

CORNERSTONE PDK Monitor — User Manual (v1.1.0)

A complete guide to using the dashboard: browsing, fetching, contributing, and storing large datasets in the cloud.

1. What it is

The CORNERSTONE PDK Monitor is a single, self-contained HTML application for managing and sharing integrated-photonics **Process Design Kits**. It organises **building blocks (BBs)** — e.g. waveguides, MMIs, rings, modulators — into **platforms** (e.g. SOI 220 nm) and **tabs**. Each BB carries:

- **Layout** (GDS / OAS / SVG) and a sketch image
- **S-matrix** datasets (S-parameters)
- **Tests** (measurement / characterisation data)
- **Fabrication** notes and data
- **Scripts** (Python / tools)
- **Documents** (datasheets, manuals, PDFs)

Everything lives in one `.html` file. Data is stored locally in your browser (IndexedDB) and shared through GitHub and/or cloud storage.

2. Modes and access

The dashboard has three modes, selected from the header:

- **Read-only** (default): browse everything, fetch data, leave comments. No sign-in.
- **User-admin**: contribute your own BBs and files; publish your slice. Sign in with a GitHub identity.
- **Admin**: full control of the dashboard — curate the contributor list, manage platforms, publish the canonical dashboard.

Read-only viewers can do more than before: **fetch from GitHub and cloud, and add comments**, without signing in.

2.1 Header layout

The header is grouped into a few compact menus to keep it tidy:

- ⓘ **About** — version/contact/logo/links, plus quick access to ⓘ **Contributors**, ⓘ **Help & FAQ**, and the ⓘ **Activity log**.
- ⓘ **Distribution** — fetch/publish the shared dashboard, and the duplicate resolvers.
- ⚙️ **Configuration** — ⓘ **GitHub account**, ▲ **Cloud storage**, ⓘ **Manage lists**, and the ⓘ **Status manager**. A small dot on this button shows GitHub connection state (solid = a token is set).
- **Analyse ▼** — ⓘ **Cross-platform analysis** and ⓘ **AI Insight**.
- ⬇️ **Download ▼** — scoped BB bundle, **Export BBs (xlsx)**, and **Backup (.json / .zip)**.
- ⬇️ **Import ▼** — import a backup/dashboard.json, or run a GitHub fetch (admins).

Every modal that opens another window shows a ◀ **Back** button next to Close, so you can step back through nested windows. The ^ **Min** toggle collapses the header to just the filters and Cleanup.

3. Getting the data — Distribution

The dashboard ships as an empty shell; the live PDK comes from GitHub.

1. Header → ⓘ **Distribution**.
2. ⓘ **Fetch all enabled sources** pulls the published `dashboard.json` plus every contributor's `users/<id>/dashboard.json` slice (auto-pull is on by default; untick *"Also auto-pull user slices"* for a clean admin-only snapshot).
3. Choose **Merge into my dashboard** (additive) or **Replace whole dashboard**. A local backup `.json` is downloaded first.

This works for public repos with **no GitHub account and no token**. Large `dashboard.json` files (over 1 MB) are fetched via the raw host automatically.

4. Importing a platform from GitHub

On a platform column header, click ⬇️ **Import platform**:

- Add one or more GitHub source URLs (admins/user-admins); read-only viewers run against admin-saved URLs.
- Pick which sections to fetch (Layout, S-matrix, Tests, Fab, Scripts, Documents).

- The importer walks `components/` and creates one BB per `*.gds`.

If GitHub's unauthenticated rate limit is hit, the importer automatically falls back to a single repo-tree listing so the import still completes.

5. Cloud data storage (large datasets)

Why: GitHub repos should stay small (well under 5 GB). Keep the catalogue in GitHub and the heavy raw datasets in cloud storage; the dashboard fetches them on demand and caches them locally. Only a small pointer travels in `dashboard.json`.

5.1 Where to set cloud links

- **Per file:**  `Link` on a dataset/test card — paste a direct URL or a SharePoint/OneDrive share link.
- **Per folder:**  `Cloud base URL` on an S-matrix / Tests / Fab folder — files resolve as `<base>/<filename>`.
- **Per section:** the  `Cloud links` button on each BB tab (next to  `GitHub fetch`).
- **Whole BB:** the  `Cloud data` panel on the **Details** tab →  `Fetch all cloud data for this BB` runs every section's links at once.

All of these are available to **read-only viewers** for fetching; only admins/user-admins can change the links.

5.2 Fetching

- A cloud-backed file shows  `Fetch`. Click it (or Download / Plot) and the bytes stream in and cache locally; then View raw / Plot / Download work as normal.
- **Default (no tick)** fetches each section's known files individually — including the common **single-file layout** where a section's data is one file named after the BB (e.g. `sparams/<bb>.csv`). It tries the base both as a file stem (`<base>.csv`) and as a folder (`<base>/<bb>.csv`).
- Tick **Fetch everything** (off by default, available on the per-tab  `Cloud` window and the Details panel) to instead **list the whole cloud folder, including subfolders**, recreate that structure under a clearly-marked  `Cloud` folder (with nested subfolders), and import **only files not already present** (so GitHub-fetched files aren't duplicated). Supported for **S3** (needs `s3:ListBucket`; recursive + paginated) and **SharePoint/OneDrive** (walks subfolders via Graph).
- Use  **Test** before committing a link — it reports whether the URL is reachable from the browser (CORS) and, for a folder, how many files it contains plus the total size. It also surfaces the real S3 error (e.g. `AccessDenied`) instead of a bare 403.

5.2a Uploading to cloud (admins)

The dashboard can also **push files into your own bucket**, not just read from it. In  **Cloud storage** (Configuration), admins/maintainers see  **Configure cloud upload**:

- **AWS S3** (and S3-compatible R2 / B2 / MinIO): region, bucket, optional custom endpoint, access key ID, secret, optional STS session token, and a default key prefix. Uploads are signed in-browser with **AWS Signature V4**.
- **Generic PUT endpoint** for an internal server or pre-signed prefix.
- Credentials are stored **only in this browser** (localStorage) — never written into the dashboard or published. Use  **Test write** to confirm credentials + CORS allow **PUT**.

Then every S-matrix / Tests / Fabrication folder gets an admin-only  **Cloud upload** button that pushes that folder and its subfolders up, recreating the structure as key prefixes. Files without local bytes (un-fetched cloud stubs) are skipped. Your bucket's CORS must allow **PUT** and the IAM identity needs `s3:PutObject`.

5.3 Provider notes

The fetch is a browser request, so the host **must allow cross-origin (CORS) reads**.

Amazon S3 1. Bucket → **Permissions** → **CORS** → paste: `json [{ "AllowedOrigins": ["https://YOUR-DASHBOARD-ORIGIN"], "AllowedMethods": ["GET", "HEAD"], "AllowedHeaders": ["*"], "ExposeHeaders": ["Content-Length", "Content-Range", "ETag", "Accept-Ranges"], "MaxAgeSeconds": 3000 }]` Replace the origin with where the dashboard is served (scheme + host, no path). 2. Make objects readable: either **public-read** (bucket policy granting `s3:GetObject`) or use **pre-signed URLs**. To auto-list a folder, also allow `s3:ListBucket`. 3. A 403 after CORS is set means the object isn't readable (not a CORS problem) — fix access or use a pre-signed URL.

Azure Blob / GCS / R2 / B2 / MinIO — enable CORS for your origin and use public or pre-signed URLs. Same pattern.

Dropbox / Google Drive — paste the normal share link; the dashboard converts it to a direct-download form. Note some consumer links don't send CORS headers and may not work in-browser — object storage is more reliable.

SharePoint / OneDrive (private org data) 1. Header →  **Cloud** → expand "Connect private SharePoint / OneDrive". 2. One-time Azure setup: register a **Single-page application**, set the redirect URI to the dashboard's URL (shown in the modal), add delegated Graph permissions `Files.Read.All + Sites.Read.All`, paste the **Client ID**, Save. 3. **Sign in**. Then paste SharePoint/OneDrive "Copy link" URLs as file or folder links. 4. Requires the dashboard served over **https** (OAuth can't run from `file://`).

5.4 Two-source model — GitHub + a cloud mirror

A platform can be fed by **two sources at once**. **GitHub is the principal source** — it fetches exactly the sections you tick in the import, as always. A **cloud mirror** of the same repo is an **assisting source** for the heavy data (S-matrix, tests, fabrication, scripts, documents): enabling it does **not** change or replace what GitHub pulls — it only *adds* cloud links you can fetch on demand. Leave the cloud field blank for a GitHub-only import.

Set it up during import. In **↓ Import platform** there is a **Cloud mirror** field with a **Test** button and a checkbox **"Populate cloud links on each BB from this mirror"** (off by default). When ticked:

- Only the **layout** (`.gds` / `.oas` / `.svg`) is fetched from GitHub.
- Every BB's heavy tabs are given cloud links that mirror the repo folders — by convention:
- S-matrix → `<mirror>/sparams/<bb>/`
- Tests → `<mirror>/characterisation/<bb>/`
- Fabrication → `<mirror>/fabrication/<bb>/`
- Scripts → `<mirror>/scripts/<bb>/`
- Documents → `<mirror>/documents/<bb>/`
- The Details **main link** points at the mirror root.
- **Nothing is downloaded.** The links are distributed across the BBs' Details panel and per-tab **Cloud links**; fetch the data on demand (per tab, or the Details *Fetch all*, or *Fetch everything* to auto-list a folder).

So GitHub stays small and fast (catalogue + layout) while the gigabytes of measurement/simulation/fabrication data live in the cloud and stream in only when someone opens them.

Keep it in sync. Each platform header has a **Cloud sync** button (next to **Sync**) — it re-distributes the cloud links across all the platform's BBs from the configured mirror, without downloading. Use it after adding BBs or changing the mirror.

Real folder names. When populating from a mirror, the dashboard inspects the GitHub repo's actual top-level folders and mirrors those exact names (e.g. if the repo uses `test/`, the cloud links point at `test/`, not a renamed `characterisation/`).

Design manuals from cloud. The platform's **Design manuals** window has a **Cloud fetch** button alongside **Sync from GitHub** — point it at a cloud folder (S3 / SharePoint) to pull the manuals from there. Available to read-only viewers once an admin sets the URL.

5.5 Bulk selection

Every platform column header has a **Select all** toggle. It ticks (or clears) every building block shown in that column — respecting the current filter — so you can **Move** or **Download** them in bulk. You can also tick

individual BBs with the checkbox beside each card.

File-level selection. Inside a BB, every file (in any tab, including subfolders) has a checkbox. Each **folder** carries its own ☒ **Select all** and ☐ **Delete selected** buttons right next to its **+ Add comment** button, so you can tick that folder's files — for example the cloud-imported test files — and delete just them, without affecting other folders. The flat **Scripts** tab has a single Select-all / Delete bar at the top.

The mirror mapping assumes the CORNERSTONE repo convention above. If your cloud layout differs, adjust any BB's per-tab **Cloud base URL** manually.

5.6 Uploads view & layout files

The **Uploads** window lists every uploaded folder across all BBs. **Layout files (GDS / OAS / SVG)** now appear here under a **Layout** section for each BB — traceable however they arrived (manual upload, GitHub fetch, or import). Every file row has a **delete** button (admins): deleting a layout entry clears that BB's layout slot; deleting any other file removes it (including from nested subfolders).

5.7 Finding duplicates

Distribution → **Resolve duplicates** folds BBs that share a name. **Resolve duplicate files** scans every BB for files that appear more than once (same name/size) across all tabs, subfolders, and uploaded files — tick the redundant copy and delete it while keeping the other. A backup `.json` downloads before any change.

6. Comments

Every S-matrix folder, test folder, fabrication sub-tab, and script has a **Comments** thread, and each BB's **Details** tab has **+ Add note**. Any contributor (including read-only) can add one — it's stamped with their name and date. Only the author or an admin can delete it.

Comments and Details notes **travel with publish/merge** and are **unioned across contributors**, so when you fetch everyone's repos you see everyone's comments (de-duplicated by id), without any file contents riding along.

7. Contributing and publishing

- **Add BBs / files** in user-admin or admin mode (drag-drop, GitHub fetch, or cloud fetch).
- **Publish** via Distribution: a user-admin publishes a slice to `users/<id>/dashboard.json`; an admin

publishes the canonical `dashboard.json`.

- **The published `dashboard.json` is a catalogue — metadata only.** File bytes (S-matrix / tests / fab / uploads) and their parsed arrays are stripped on publish regardless of size; only filenames, sizes, cloud/GitHub pointers, comments and notes travel. This keeps repos small even as data and discussion grow — adopters re-hydrate bytes on demand from cloud/GitHub. (Small documents/YAML descriptors under 512 KB still round-trip.) Your local copy keeps full bytes.
 - The **contributor list** (admin-managed) is the single source of who's who; edit it from About → [?](#) **Contributors**.
-

8. Tips & troubleshooting

- **"Nothing to fetch"** in Distribution → check the source path; for contributor slices ensure *auto-pull user slices* is ticked (it is by default).
 - **403 from S3** → the object isn't readable; make it public or use a pre-signed URL. CORS only governs whether the browser can read the response.
 - **SharePoint sign-in fails** → confirm the dashboard is on https and the Azure redirect URI matches exactly.
 - **Cloud file won't fetch but Test says reachable** → check the filename matches the object key under the base URL. For single-file S-matrix layouts, the file is fetched as `<base>.csv`; tick **Fetch everything** if your data is in nested folders instead.
 - **Cloud upload fails** → the bucket CORS must allow `PUT` for this origin and the key/secret's IAM identity needs `s3:PutObject`. Use [?](#) **Test write** to diagnose.
 - **Storage** → fetched cloud bytes cache in your browser (IndexedDB). Use the storage diagnostics in the header to monitor usage.
-

CORNERSTONE PDK Monitor — first designed and released by Emre Kaplan, University of Southampton.

Maintained by the CORNERSTONE PDK team.